

FISM 2026 - Lezione 03/06/2026

Soluzioni degli esercizi

Questo PDF contiene soluzioni ed esempi di correzione. Le soluzioni non sono uniche: in HTML/CSS possono esistere varianti corrette.

Indice degli esercizi

N.	Sezione	Titolo	Diff.	Tempo
01	A. Repository e consegna	Audit iniziale della repository	Base	15 min
02	A. Repository e consegna	Ricostruire una repository disordinata	Base	20 min
03	A. Repository e consegna	README minimo ma utile	Base	25 min
04	A. Repository e consegna	CHANGELOG della giornata	Base	10 min
05	A. Repository e consegna	Troubleshooting realistico	Intermedio	20 min
06	B. Tema, HTML e contenuto	Scegliere un tema non documentale	Base	15 min
07	B. Tema, HTML e contenuto	Homepage semantica generica	Base	25 min
08	B. Tema, HTML e contenuto	Correggere link e percorsi relativi	Base	20 min
09	B. Tema, HTML e contenuto	Sito multipagina minimo	Intermedio	30 min
10	B. Tema, HTML e contenuto	Immagini accessibili e ordinate	Base	20 min
11	C. CSS intermedio	Pagina leggibile con max-width	Base	20 min
12	C. CSS intermedio	Spaziature coerenti	Base	20 min
13	C. CSS intermedio	Card con Flexbox	Base	25 min
14	C. CSS intermedio	Responsive: card impilate su mobile	Intermedio	20 min
15	C. CSS intermedio	Galleria con CSS Grid	Intermedio	30 min
16	C. CSS intermedio	Hero e call to action	Base	25 min
17	C. CSS intermedio	Debug di navbar rotta	Intermedio	20 min
18	C. CSS intermedio	Revisione critica del design	Intermedio	25 min
19	D. Bootstrap ragionato	Collegare Bootstrap nel modo corretto	Base	15 min
20	D. Bootstrap ragionato	Container e spacing utilities	Base	20 min
21	D. Bootstrap ragionato	Grid Bootstrap con card	Intermedio	30 min
22	D. Bootstrap ragionato	Colori semantici Bootstrap	Base	20 min
23	D. Bootstrap ragionato	Content Bootstrap: testo, immagini, tabelle	Intermedio	30 min
24	D. Bootstrap ragionato	Navbar Bootstrap spiegata	Intermedio	35 min
25	D. Bootstrap ragionato	Accordion FAQ Bootstrap	Intermedio	30 min
26	D. Bootstrap ragionato	Refactoring con utilities	Avanzato	35 min
27	E. GitHub Pages	Preparare la pubblicazione	Base	15 min
28	E. GitHub Pages	Diagnosticare Pages che non funziona	Intermedio	25 min

N.	Sezione	Titolo	Diff.	Tempo
29	E. GitHub Pages	Aggiornare README dopo pubblicazione	Base	10 min
30	E. GitHub Pages	Issue di pubblicazione	Intermedio	20 min
31	F. JSON/API opzionale	Progettare data.json	Intermedio	25 min
32	F. JSON/API opzionale	Renderizzare card da data.json	Avanzato	40 min
33	F. JSON/API opzionale	Documentare dati/API	Intermedio	25 min
34	F. JSON/API opzionale	Fallback se il JSON non si carica	Avanzato	30 min
35	G. Sprint finale	Sprint finale di consegna	Finale	45-60 min

A. Repository e consegna

Esercizio 01 - Audit iniziale della repository

Difficoltà: Base | Tempo stimato: 15 min

Obiettivo. Capire se la repository è pronta per diventare un progetto consegnabile.

Consegna.

- 1 Apri la repository del progetto.
- 2 Controlla se esistono i file obbligatori.
- 3 Scrivi una lista di problemi da correggere.
- 4 Fai commit della checklist in README.md o in docs/checklist.md.

File coinvolti. README.md, docs/checklist.md opzionale

Vincoli.

- Non correggere ancora i problemi: prima devi individuarli.
- La checklist deve essere leggibile da un compagno.

Soluzione / esempio di soluzione

Checklist repository

- [x] README.md presente
- [x] index.html presente
- [x] style.css presente
- [] LICENSE mancante
- [] CHANGELOG.md mancante
- [x] cartella docs/ presente
- [] docs/troubleshooting.md mancante
- [x] assets/immagini/ presente

Problemi da correggere

1. Aggiungere LICENSE.
2. Aggiungere CHANGELOG.md.
3. Creare docs/troubleshooting.md.
4. Controllare i link interni nel README.
5. Verificare che le immagini siano in assets/immagini/.

Comandi utili

```
git status
git add README.md docs/checklist.md
git commit -m "aggiunge checklist repository"
git push
```

Esercizio 02 - Ricostruire una repository disordinata

Difficoltà: Base | Tempo stimato: 20 min

Obiettivo. Sistemare una struttura di progetto confusa.

Consegna.

- 1 Parti dalla struttura sbagliata mostrata sotto.
- 2 Rinomina i file in modo corretto.
- 3 Crea le cartelle mancanti.
- 4 Sposta immagini e documentazione nelle cartelle giuste.

File coinvolti. tutta la repository

Vincoli.

- Usa nomi minuscoli per i file HTML/CSS.
- Evita spazi nei nomi delle immagini.
- Non cancellare contenuto senza motivo.

Soluzione / esempio di soluzione

Struttura corretta:

```
``text
progetto/
|-- README.md
|-- LICENSE
|-- CHANGELOG.md
|-- index.html
|-- style.css
|-- assets/
|   |-- immagini/
|   |-- screenshot-finale.png
|-- docs/
|   |-- quick-start.md
|   |-- guida-utente.md
|   |-- installazione.md
|   |-- faq.md
|   |-- troubleshooting.md
|   |-- architettura.md
...

```

Comandi utili

```
mv Index.html index.html
mv STYLE.CSS style.css
mv readme.txt README.md
mkdir -p assets/immagini docs
mv "immagini varie/Screenshot finale.PNG" assets/immagini/screenshot-finale.png
touch LICENSE CHANGELOG.md
touch docs/quick-start.md docs/guida-utente.md docs/installazione.md docs/faq.md
docs/troubleshooting.md docs/architettura.md
git add .
git commit -m "riorganizza struttura progetto"
git push

```

Esercizio 03 - README minimo ma utile

Difficolta: Base | Tempo stimato: 25 min

Obiettivo. Scrivere un README che orienti subito chi apre la repository.

Consegna.

- 1 Crea o migliora README.md.
- 2 Inserisci descrizione, funzionalita, struttura, documentazione, sito online e licenza.
- 3 Lascia il link GitHub Pages come placeholder se non e ancora attivo.

File coinvolti. README.md

Vincoli.

- Non scrivere un testo generico tipo "questo e il progetto".
- Ogni funzionalita deve avere una descrizione concreta.

Soluzione / esempio di soluzione

Catalogo Film

Catalogo Film e un sito statico HTML/CSS per presentare una selezione di film

consigliati.
 Il progetto è stato realizzato per il corso FISM 2026.

Funzionalità

- **Catalogo a card**: ogni film viene mostrato con titolo, immagine e breve descrizione.
- **Sezioni responsive**: il layout si adatta a schermi piccoli.
- **FAQ**: una sezione risponde alle domande più comuni.
- **Pubblicazione online**: il sito è pubblicato con GitHub Pages.

Sito online

Il sito è disponibile qui: <https://nomeutente.github.io/nome-progetto/>

Struttura repository

```
| Percorso | Scopo |
|---|---|
| `index.html` | Homepage del sito |
| `style.css` | Stili CSS personalizzati |
| `assets/immagini/` | Immagini usate nel sito |
| `docs/` | Documentazione del progetto |
```

Documentazione

- [Quick start](docs/quick-start.md)
- [Guida utente](docs/guida-utente.md)
- [Installazione](docs/installazione.md)
- [FAQ](docs/faq.md)
- [Troubleshooting](docs/troubleshooting.md)
- [Architettura](docs/architettura.md)

Licenza

Questo progetto è distribuito con licenza MIT.

Esercizio 04 - CHANGELOG della giornata

Difficoltà: Base | Tempo stimato: 10 min

Obiettivo. Registrare le modifiche principali fatte durante la lezione.

Consegna.

- 1 Crea o aggiorna CHANGELOG.md.
- 2 Aggiungi una versione 0.2.0 per il lavoro del 03/06.
- 3 Dividi le modifiche in Aggiunto, Modificato e Corretto.

File coinvolti. CHANGELOG.md

Vincoli.

- Non scrivere solo "aggiornato progetto".
- Le voci devono essere verificabili nella repository.

Soluzione / esempio di soluzione

```
# Changelog

## 0.2.0 - 2026-06-03

### Aggiunto
- Layout migliorato della homepage.
- Sezione con card per i contenuti principali.
- Footer del sito.
- Pubblicazione con GitHub Pages.

### Modificato
- Aggiornato README con link al sito online.
- Migliorata organizzazione delle immagini in `assets/immagini/`.
```

```
### Corretto
- Sistemati percorsi relativi di CSS e immagini.
- Corretto nome del file `index.html`.
```

Esercizio 05 - Troubleshooting realistico

Difficoltà: Intermedio | Tempo stimato: 20 min

Obiettivo. Scrivere una pagina di troubleshooting utile per problemi veri del sito.

Consegna.

- 1 Crea docs/troubleshooting.md.
- 2 Inserisci almeno 4 problemi comuni.
- 3 Per ogni problema scrivi causa possibile, controllo e soluzione.

File coinvolti. docs/troubleshooting.md

Vincoli.

- I problemi devono essere realistici per HTML/CSS/GitHub Pages.
- Non copiare frasi vaghe come "controllare tutto".

Soluzione / esempio di soluzione

```
# Troubleshooting

## Il CSS non viene caricato

### Possibile causa
Il percorso del file CSS è sbagliato oppure il file non si chiama esattamente
`style.css`.

### Controlli
- Verificare che `style.css` sia nella stessa cartella di `index.html`.
- Verificare che nel file HTML ci sia `
```

B. Tema, HTML e contenuto

Esercizio 06 - Scegliere un tema non documentale

Difficoltà: Base | Tempo stimato: 15 min

Obiettivo. Definire un sito generico, non per forza legato alla documentazione.

Consegna.

- 1 Scegli un tema per il sito finale.
- 2 Compila una tabella con target, sezioni, immagini e dati.
- 3 Aggiorna README o docs/architettura.md.

File coinvolti. README.md oppure docs/architettura.md

Vincoli.

- Non scegliere "sito documentazione" se non è il tema che vuoi davvero.
- Il tema deve essere realizzabile in HTML/CSS statico.

Soluzione / esempio di soluzione

```
| Campo | Risposta |
|---|---|
| Tema sito | Sito evento musicale |
| Utente destinatario | Persona interessata al programma dell'evento |
| Sezioni principali | Hero, programma, artisti, luogo, FAQ, contatti |
| Immagini necessarie | Foto palco, immagini artisti, mappa luogo |
| Dati eventuali | `data.json` con orari e artisti |
| Funzionalità extra | Filtro per giorno o categoria |
```

Esercizio 07 - Homepage semantica generica

Difficoltà: Base | Tempo stimato: 25 min

Obiettivo. Costruire una struttura HTML sensata per un sito generico.

Consegna.

- 1 Crea una homepage per il tema scelto.
- 2 Usa header, nav, main, section e footer.
- 3 Inserisci almeno 4 sezioni reali.

File coinvolti. index.html

Vincoli.

- Non usare solo div.
- Ogni section deve avere un titolo h2.
- I link della navbar devono puntare alle sezioni.

Soluzione / esempio di soluzione

```
<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Festival Note d'Estate</title>
  <link rel="stylesheet" href="style.css">
```

```

</head>
<body>
  <header class="hero" id="home">
    <h1>Festival Note d'Estate</h1>
    <p>Tre giorni di musica, incontri e spettacoli dal vivo.</p>
    <a class="button" href="#programma">Scopri il programma</a>
  </header>

  <nav class="navbar">
    <a href="#programma">Programma</a>
    <a href="#artisti">Artisti</a>
    <a href="#luogo">Luogo</a>
    <a href="#faq">FAQ</a>
  </nav>

  <main>
    <section id="programma">
      <h2>Programma</h2>
      <p>Concerti e attività distribuite su tre giornate.</p>
    </section>

    <section id="artisti">
      <h2>Artisti</h2>
      <p>Una selezione di artisti locali e ospiti nazionali.</p>
    </section>

    <section id="luogo">
      <h2>Luogo</h2>
      <p>L'evento si svolge nel parco cittadino.</p>
    </section>

    <section id="faq">
      <h2>FAQ</h2>
      <p>Risposte alle domande più frequenti.</p>
    </section>
  </main>

  <footer>
    <p>Progetto FISM 2026</p>
  </footer>
</body>
</html>

```

Esercizio 08 - Correggere link e percorsi relativi

Difficoltà: Base | Tempo stimato: 20 min

Obiettivo. Imparare a diagnosticare immagini e link rotti.

Consegna.

- 1 Correggi i percorsi sbagliati nel frammento HTML.
- 2 Assumi che index.html sia nella root.
- 3 Assumi che le immagini siano in assets/immagini/.

File coinvolti. index.html

Vincoli.

- Usa percorsi relativi.
- Evita spazi nei nomi file.
- Usa testo alternativo descrittivo.

Soluzione / esempio di soluzione

Se il file immagine si chiamava davvero `Home Page.png`, va rinominato in `homepage.png` e poi va aggiornato il riferimento HTML.

```

<link rel="stylesheet" href="style.css">

<a href="docs/guida-utente.md">Guida utente</a>

```


Esercizio 09 - Sito multipagina minimo

Difficoltà: Intermedio | Tempo stimato: 30 min

Obiettivo. Creare un sito con più pagine mantenendo una navigazione coerente.

Consegna.

- 1 Crea index.html, catalogo.html e contatti.html.
- 2 Inserisci la stessa navbar in tutte le pagine.
- 3 Collega lo stesso style.css.
- 4 Aggiungi un link alla repository GitHub.

File coinvolti. index.html, catalogo.html, contatti.html, style.css

Vincoli.

- Tutte le pagine devono essere raggiungibili dalla navbar.
- Non usare percorsi assoluti locali tipo C:\Users\...

Soluzione / esempio di soluzione

La stessa navbar può essere copiata nelle tre pagine. In un corso breve va bene duplicarla. In progetti più avanzati si userebbero template o componenti.

```
<nav class="navbar">
  <a href="index.html">Home</a>
  <a href="catalogo.html">Catalogo</a>
  <a href="contatti.html">Contatti</a>
  <a href="https://github.com/nomeutente/nome-progetto">GitHub</a>
</nav>
```

Esercizio 10 - Immagini accessibili e ordinate

Difficoltà: Base | Tempo stimato: 20 min

Obiettivo. Usare immagini in modo ordinato e con alt text sensato.

Consegna.

- 1 Inserisci 3 immagini nel sito.
- 2 Spostale in assets/immagini/.
- 3 Rinominalle con nomi chiari.
- 4 Scrivi alt text descrittivi.

File coinvolti. index.html, assets/immagini/

Vincoli.

- No nomi tipo Screenshot 2026-06-03 alle 12.31.png.
- No alt="immagine".

Soluzione / esempio di soluzione

```



```

C. CSS intermedio

Esercizio 11 - Pagina leggibile con max-width

Difficoltà: Base | Tempo stimato: 20 min

Obiettivo. Evitare pagine larghe e testo difficile da leggere.

Consegna.

- 1 Aggiungi regole CSS globali.
- 2 Centra il contenuto principale.
- 3 Rendi le immagini responsive.

File coinvolti. style.css

Vincoli.

- Non usare width fissa in pixel sul body.
- Il sito deve adattarsi a schermi piccoli.

Soluzione / esempio di soluzione

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  line-height: 1.6;
  color: #222;
  background: #f5f5f5;
}

main {
  max-width: 1100px;
  margin: 0 auto;
  padding: 24px;
}

img {
  max-width: 100%;
  height: auto;
}
```

Esercizio 12 - Spaziature coerenti

Difficoltà: Base | Tempo stimato: 20 min

Obiettivo. Eliminare l'effetto pagina amatoriale dovuto a spazi casuali.

Consegna.

- 1 Definisci spaziature coerenti per header, section, card e footer.
- 2 Usa valori ricorrenti: 8, 16, 24, 32, 48, 64 px.
- 3 Evita numeri casuali.

File coinvolti. style.css

Vincoli.

- Non usare margin diversi a caso per ogni elemento.
- Le sezioni devono essere chiaramente separate.

Soluzione / esempio di soluzione

```
header {  
  padding: 64px 24px;  
}  
  
section {  
  margin-bottom: 32px;  
  padding: 24px;  
}  
  
.card {  
  padding: 20px;  
  margin-bottom: 20px;  
}  
  
footer {  
  padding: 24px;  
}
```

Esercizio 13 - Card con Flexbox

Difficoltà: Base | Tempo stimato: 25 min

Obiettivo. Creare una riga di card ordinate e flessibili.

Consegna.

- 1 Crea un contenitore .cards.
- 2 Inserisci almeno 3 .card.
- 3 Usa Flexbox per disporle in riga.

File coinvolti. index.html, style.css

Vincoli.

- Le card devono avere uguale larghezza su desktop.
- Devono avere spazio tra loro.

Soluzione / esempio di soluzione

```
.cards {  
  display: flex;  
  gap: 20px;  
}  
  
.card {  
  flex: 1;  
  padding: 20px;  
  background: white;  
  border: 1px solid #ddd;  
  border-radius: 12px;  
}
```

Esercizio 14 - Responsive: card impilate su mobile

Difficoltà: Intermedio | Tempo stimato: 20 min

Obiettivo. Rendere le card leggibili anche da telefono.

Consegna.

- 1 Aggiungi una media query.

- 2 Sotto 768px, disponi navbar e card in colonna.
- 3 Verifica ridimensionando il browser.

File coinvolti. style.css

Vincoli.

- Non creare due versioni diverse della pagina.
- Usa CSS responsive.

Soluzione / esempio di soluzione

```
@media (max-width: 768px) {  
  .navbar,  
  .cards {  
    flex-direction: column;  
  }  
  
  main {  
    padding: 16px;  
  }  
}
```

Esercizio 15 - Galleria con CSS Grid

Difficoltà: Intermedio | Tempo stimato: 30 min

Obiettivo. Usare Grid per una galleria responsive di immagini o elementi.

Consegna.

- 1 Crea una sezione galleria.
- 2 Usa CSS Grid con auto-fit o auto-fill.
- 3 Fai in modo che le colonne si adattino allo spazio disponibile.

File coinvolti. index.html, style.css

Vincoli.

- Non usare tre colonne fisse se poi su mobile si rompe.
- Le immagini devono restare dentro le card.

Soluzione / esempio di soluzione

```
.gallery {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));  
  gap: 20px;  
}  
  
.gallery-item {  
  background: white;  
  border-radius: 12px;  
  overflow: hidden;  
  border: 1px solid #ddd;  
}  
  
.gallery-item img {  
  width: 100%;  
  display: block;  
}
```

Esercizio 16 - Hero e call to action

Difficoltà: Base | Tempo stimato: 25 min

Obiettivo. Creare una sezione iniziale chiara e orientata all'azione.

Consegna.

- 1 Crea una hero con titolo, sottotitolo e bottone.
- 2 Il bottone deve portare alla sezione principale.
- 3 La hero deve avere contrasto sufficiente.

File coinvolti. index.html, style.css

Vincoli.

- Non usare un bottone senza destinazione.
- Il testo deve spiegare subito il sito.

Soluzione / esempio di soluzione

```
.hero {  
  padding: 72px 24px;  
  text-align: center;  
  background: #222;  
  color: white;  
}  
  
.hero p {  
  max-width: 650px;  
  margin: 0 auto 24px;  
}  
  
.button {  
  display: inline-block;  
  padding: 12px 18px;  
  border-radius: 8px;  
  background: #0d6efd;  
  color: white;  
  text-decoration: none;  
  font-weight: bold;  
}
```

Esercizio 17 - Debug di navbar rotta

Difficoltà: Intermedio | Tempo stimato: 20 min

Obiettivo. Correggere una navbar con spaziature e link disordinati.

Consegna.

- 1 Parti dal CSS sbagliato.
- 2 Rendi la navbar orizzontale su desktop.
- 3 Rendila verticale su mobile.
- 4 Rimuovi sottolineatura dai link.

File coinvolti. style.css

Vincoli.

- Non usare margin enorme sui link.
- Usa gap sul contenitore.

Soluzione / esempio di soluzione

```
.navbar {  
  display: flex;  
  justify-content: center;  
  gap: 16px;
```

```

padding: 16px;
background: #222;
}

.navbar a {
color: white;
text-decoration: none;
font-weight: bold;
}

.navbar a:hover {
text-decoration: underline;
}

@media (max-width: 600px) {
.navbar {
flex-direction: column;
align-items: center;
}
}

```

Esercizio 18 - Revisione critica del design

Difficoltà: Intermedio | Tempo stimato: 25 min

Obiettivo. Saper valutare se un sito statico è presentabile.

Consegna.

- 1 Scambia il sito con un compagno.
- 2 Compila una review tecnica.
- 3 Indica almeno 3 problemi e 3 miglioramenti concreti.

File coinvolti. docs/review.md oppure issue GitHub

Vincoli.

- La review deve essere specifica.
- Non basta scrivere "bello" o "brutto".

Soluzione / esempio di soluzione

```

## Review sito

### Problemi trovati

1. La navbar non è visibile su sfondo scuro per contrasto insufficiente.
2. Le immagini della galleria hanno dimensioni incoerenti.
3. Su mobile le card restano troppo strette.

### Miglioramenti proposti

1. Usare testo bianco nella navbar e aumentare il padding.
2. Applicare `width: 100%` e `height: auto` alle immagini.
3. Aggiungere una media query per impilare le card sotto 768px.

### Valutazione finale

Il sito è funzionante, ma deve migliorare leggibilità e responsive design.

```

D. Bootstrap ragionato

Esercizio 19 - Collegare Bootstrap nel modo corretto

Difficoltà: Base | Tempo stimato: 15 min

Obiettivo. Capire ordine e ruolo dei file Bootstrap e CSS personalizzato.

Consegna.

- 1 Aggiungi Bootstrap via CDN.
- 2 Aggiungi il tuo style.css dopo Bootstrap.
- 3 Spiega in un commento perché style.css va dopo.

File coinvolti. index.html

Vincoli.

- Non rimuovere il tuo CSS.
- Non inserire il JS Bootstrap dentro head se non serve.

Soluzione / esempio di soluzione

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Nome progetto</title>

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    rel="stylesheet">

  <!-- Il CSS personalizzato va dopo Bootstrap per poter sovrascrivere alcune regole.
  -->
  <link rel="stylesheet" href="style.css">
</head>
<body>
  ...
  <script
    src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
```

Esercizio 20 - Container e spacing utilities

Difficoltà: Base | Tempo stimato: 20 min

Obiettivo. Usare Bootstrap non a caso, ma per controllare larghezza e spaziature.

Consegna.

- 1 Avvolgi il contenuto principale in .container.
- 2 Usa my-5, mb-4, p-4 dove ha senso.
- 3 Annota il significato di almeno 3 classi usate.

File coinvolti. index.html

Vincoli.

- Non mettere classi Bootstrap casuali.
- Ogni classe utility scelta deve avere uno scopo.

Soluzione / esempio di soluzione

Esempi di significato:

- `container`: centra il contenuto e limita la larghezza.
- `my-5`: aggiunge margine verticale sopra e sotto.
- `mb-4`: aggiunge margine sotto al titolo.
- `p-4`: aggiunge padding interno.
- `bg-light`: applica uno sfondo chiaro.
- `rounded`: arrotonda i bordi.

```
<main class="container my-5">
```

```

<section class="mb-5">
  <h2 class="mb-4">Servizi</h2>
  <p class="lead">Una panoramica dei servizi principali offerti dal sito.</p>
</section>

<section class="p-4 bg-light rounded">
  <h2 class="mb-3">Informazioni</h2>
  <p class="mb-0">Questa sezione usa padding, sfondo chiaro e bordi arrotondati.</p>
</section>
</main>

```

Esercizio 21 - Grid Bootstrap con card

Difficoltà: Intermedio | Tempo stimato: 30 min

Obiettivo. Costruire una sezione a card responsive usando la griglia.

Consegna.

- 1 Crea una sezione con 3 card.
- 2 Usa row, g-4 e col-md-4.
- 3 Ogni card deve avere titolo, testo e bottone.

File coinvolti. index.html

Vincoli.

- Non usare tabelle per fare layout.
- Le card devono impilarsi automaticamente su mobile.

Soluzione / esempio di soluzione

```

<section class="container my-5" id="servizi">
  <h2 class="mb-4">Servizi</h2>

  <div class="row g-4">
    <div class="col-md-4">
      <div class="card h-100">
        <div class="card-body">
          <h3 class="card-title">Catalogo</h3>
          <p class="card-text">Visualizza gli elementi principali del sito.</p>
          <a href="#" class="btn btn-primary">Apri</a>
        </div>
      </div>
    </div>

    <div class="col-md-4">
      <div class="card h-100">
        <div class="card-body">
          <h3 class="card-title">Categorie</h3>
          <p class="card-text">Naviga tra le sezioni disponibili.</p>
          <a href="#" class="btn btn-primary">Scopri</a>
        </div>
      </div>
    </div>

    <div class="col-md-4">
      <div class="card h-100">
        <div class="card-body">
          <h3 class="card-title">FAQ</h3>
          <p class="card-text">Leggi le risposte alle domande frequenti.</p>
          <a href="#faq" class="btn btn-primary">Leggi</a>
        </div>
      </div>
    </div>
  </div>
</section>

```

Esercizio 22 - Colori semantici Bootstrap

Difficoltà: Base | Tempo stimato: 20 min

Obiettivo. Capire che primary, success, warning e danger non sono solo colori, ma significati.

Consegna.

- 1 Crea quattro bottoni o alert con colori semantici.
- 2 Scrivi accanto a ciascuno quando usarlo.
- 3 Evita colori scelti solo per gusto personale.

File coinvolti. index.html

Vincoli.

- Usa colori Bootstrap coerenti con il messaggio.
- Non usare danger per una informazione neutra.

Soluzione / esempio di soluzione

Uso corretto:

```
- `primary`: azione o informazione principale.
- `success`: conferma positiva.
- `warning`: attenzione, rischio o controllo necessario.
- `danger`: errore o problema grave.
- `secondary`: informazione meno importante.
- `info`: nota informativa.

<div class="alert alert-primary" role="alert">
  Informazione principale sul sito.
</div>

<div class="alert alert-success" role="alert">
  Operazione completata correttamente.
</div>

<div class="alert alert-warning" role="alert">
  Attenzione: controllare che il file si chiami index.html.
</div>

<div class="alert alert-danger" role="alert">
  Errore: il file CSS non è stato trovato.
</div>
```

Esercizio 23 - Content Bootstrap: testo, immagini, tabelle

Difficoltà: Intermedio | Tempo stimato: 30 min

Obiettivo. Usare Bootstrap anche per contenuto, non solo componenti vistosi.

Consegna.

- 1 Crea una sezione con titolo, paragrafo lead, immagine responsive e tabella.
- 2 Usa classi Bootstrap per rendere leggibile il contenuto.
- 3 La tabella deve essere responsive.

File coinvolti. index.html

Vincoli.

- Non lasciare immagini enormi.
- La tabella deve stare dentro table-responsive.

Soluzione / esempio di soluzione

```
<section class="container my-5">
```

```

<h2 class="display-6">Programma evento</h2>
<p class="lead">Una panoramica sintetica delle attivita principali.</p>



<div class="table-responsive">
  <table class="table table-striped align-middle">
    <thead>
      <tr>
        <th>Orario</th>
        <th>Attivita</th>
        <th>Luogo</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>10:00</td>
        <td>Apertura</td>
        <td>Sala principale</td>
      </tr>
      <tr>
        <td>11:30</td>
        <td>Laboratorio</td>
        <td>Aula 2</td>
      </tr>
    </tbody>
  </table>
</div>
</section>

```

Esercizio 24 - Navbar Bootstrap spiegata

Difficolta: Intermedio | Tempo stimato: 35 min

Obiettivo. Usare una navbar responsive capendo le parti principali.

Consegna.

- 1 Inserisci una navbar Bootstrap.
- 2 Modifica brand e link.
- 3 Fai funzionare il menu su schermi piccoli.
- 4 Aggiungi il JS bundle Bootstrap se necessario.

File coinvolti. index.html

Vincoli.

- Il valore di data-bs-target deve corrispondere all'id del menu.
- La navbar deve puntare a sezioni esistenti.

Soluzione / esempio di soluzione

Parti principali:

- `navbar`: componente navbar.
- `navbar-expand-lg`: menu esteso da schermi grandi in su.
- `navbar-toggler`: bottone hamburger.
- `collapse navbar-collapse`: area che si apre/chiude.
- `ms-auto`: spinge i link verso destra.
- `nav-link`: stile dei link della navbar.

```

<nav class="navbar navbar-expand-lg bg-body-tertiary">
  <div class="container">
    <a class="navbar-brand" href="#home">Nome progetto</a>

    <button class="navbar-toggler" type="button"
      data-bs-toggle="collapse"
      data-bs-target="#menuPrincipale"
      aria-controls="menuPrincipale"
      aria-expanded="false"

```

```

        aria-label="Apri menu">
        <span class="navbar-toggler-icon"></span>
    </button>

    <div class="collapse navbar-collapse" id="menuPrincipale">
        <ul class="navbar-nav ms-auto">
            <li class="nav-item"><a class="nav-link" href="#servizi">Servizi</a></li>
            <li class="nav-item"><a class="nav-link" href="#galleria">Galleria</a></li>
            <li class="nav-item"><a class="nav-link" href="#faq">FAQ</a></li>
        </ul>
    </div>
</div>
</nav>

```

Esercizio 25 - Accordion FAQ Bootstrap

Difficoltà: Intermedio | Tempo stimato: 30 min

Obiettivo. Creare una FAQ interattiva con Bootstrap senza scrivere JS custom.

Consegna.

- 1 Crea una sezione FAQ con accordion.
- 2 Inserisci almeno 3 domande.
- 3 Usa testi coerenti con il tema del sito.

File coinvolti. index.html

Vincoli.

- Ogni elemento deve avere id univoco.
- data-bs-parent deve puntare all'id dell'accordion.

Soluzione / esempio di soluzione

```

<section class="container my-5" id="faq">
    <h2 class="mb-4">FAQ</h2>

    <div class="accordion" id="faqAccordion">
        <div class="accordion-item">
            <h2 class="accordion-header">
                <button class="accordion-button" type="button" data-bs-toggle="collapse"
                    data-bs-target="#faq1">
                    Il sito funziona da telefono?
                </button>
            </h2>
            <div id="faq1" class="accordion-collapse collapse show"
                data-bs-parent="#faqAccordion">
                <div class="accordion-body">
                    Sì, il layout usa griglia responsive e immagini fluide.
                </div>
            </div>
        </div>

        <div class="accordion-item">
            <h2 class="accordion-header">
                <button class="accordion-button collapsed" type="button"
                    data-bs-toggle="collapse" data-bs-target="#faq2">
                    Dove trovo il codice sorgente?
                </button>
            </h2>
            <div id="faq2" class="accordion-collapse collapse" data-bs-parent="#faqAccordion">
                <div class="accordion-body">
                    Il codice si trova nella repository GitHub del progetto.
                </div>
            </div>
        </div>
    </div>
</section>

```

Esercizio 26 - Refactoring con utilities

Difficoltà: Avanzato | Tempo stimato: 35 min

Obiettivo. Sostituire CSS ripetitivo con utility Bootstrap leggibili.

Consegna.

- 1 Parti dal frammento con classi custom.
- 2 Riscrivilo usando utilities Bootstrap dove ha senso.
- 3 Mantieni style.css solo per personalizzazioni vere.

File coinvolti. index.html, style.css

Vincoli.

- Non trasformare tutto in una zuppa illeggibile di classi.
- Usa utilities solo quando rendono il codice più chiaro.

Soluzione / esempio di soluzione

In questo caso le utility Bootstrap sostituiscono bene il CSS custom per margini, padding, sfondo e bordi arrotondati. Il CSS personalizzato va tenuto per regole specifiche del progetto, non per ogni singolo margine.

```
<section class="container my-5 p-4 bg-light rounded">
  <h2 class="mb-3">Offerta speciale</h2>
  <p class="mb-0">Testo introduttivo.</p>
</section>
```

E. GitHub Pages

Esercizio 27 - Preparare la pubblicazione

Difficoltà: Base | Tempo stimato: 15 min

Obiettivo. Verificare che il sito sia pubblicabile prima di attivare Pages.

Consegna.

- 1 Controlla che index.html sia nella root.
- 2 Controlla CSS, immagini e link.
- 3 Fai commit e push.
- 4 Scrivi nel README una sezione Sito online con placeholder.

File coinvolti. README.md, index.html, style.css

Vincoli.

- Non attivare Pages se la repo è ancora sporca.
- Non lasciare modifiche non committate.

Soluzione / esempio di soluzione

Controlli minimi:

```
```bash
git status
ls
```
```

Devono comparire almeno:

```
```text
index.html
style.css
README.md
assets/
docs/
```
```

Poi:

```
```bash
git add .
git commit -m "prepara pubblicazione GitHub Pages"
git push
```
```

README:

```
```md
Sito online
```

Il sito sarà disponibile qui: <https://nomeutente.github.io/nome-progetto/>

## Esercizio 28 - Diagnosticare Pages che non funziona

*Difficoltà: Intermedio | Tempo stimato: 25 min*

**Obiettivo.** Riconoscere gli errori più comuni nella pubblicazione.

### Consegna.

- 1 Leggi i sintomi.
- 2 Individua la causa più probabile.
- 3 Scrivi la correzione in docs/troubleshooting.md.

**File coinvolti.** docs/troubleshooting.md

### Vincoli.

- Per ogni sintomo scrivi una causa e una soluzione concreta.

### Soluzione / esempio di soluzione

Sintomo	Causa probabile	Soluzione
404	`index.html` non è nella root o Pages pubblica dalla cartella sbagliata	Controllare Settings > Pages e rinominare il file in `index.html`
CSS assente	percorso CSS sbagliato o file non pushato	usare ` <link &gt;`,="" commit="" href="style.css" poi="" push<="" rel="stylesheet" td=""/>
immagini assenti online	maiuscole/minuscole o percorso errato	controllare nome esatto e percorso `assets/immagini/...`
sito non aggiornato	commit/push mancante o cache	eseguire push, attendere, provare finestra anonima

## Esercizio 29 - Aggiornare README dopo pubblicazione

*Difficoltà: Base | Tempo stimato: 10 min*

**Obiettivo.** Rendere il link pubblico facilmente trovabile.

### Consegna.

- 1 Aggiungi link GitHub Pages al README.
- 2 Aggiungi anche link alla repository e alla documentazione.

### 3 Fai commit.

**File coinvolti.** README.md

#### **Vincoli.**

- Il link deve essere cliccabile.
- Non mettere il link solo nella descrizione della repository.

#### **Soluzione / esempio di soluzione**

## Link utili

- [Sito online](https://nomeutente.github.io/nome-progetto/)
- [Repository GitHub](https://github.com/nomeutente/nome-progetto)
- [Quick start](docs/quick-start.md)
- [Guida utente](docs/guida-utente.md)
- [FAQ](docs/faq.md)
- [Troubleshooting](docs/troubleshooting.md)

#### **Comandi utili**

```
git add README.md
git commit -m "aggiunge link sito online al README"
git push
```

## **Esercizio 30 - Issue di pubblicazione**

*Difficoltà: Intermedio | Tempo stimato: 20 min*

**Obiettivo.** Documentare un problema tecnico come issue GitHub.

#### **Consegna.**

- 1 Apri una issue per un problema GitHub Pages.
- 2 Usa titolo chiaro, descrizione, passi per riprodurre, risultato atteso e soluzione proposta.
- 3 Chiudi la issue con una pull request o un commit.

**File coinvolti.** GitHub Issues

#### **Vincoli.**

- Non scrivere solo "non funziona".
- La issue deve aiutare a correggere il problema.

#### **Soluzione / esempio di soluzione**

Titolo issue:

`Il CSS non viene caricato su GitHub Pages`

Descrizione:

Quando apro il sito pubblicato con GitHub Pages, la pagina viene mostrata senza stile.

Passaggi:

1. Aprire il link GitHub Pages.
2. Osservare che la navbar e le card non hanno stile.
3. Controllare il file `index.html`.

Risultato attuale:

Il browser non carica `style.css`.

Risultato atteso:

La pagina deve mostrare layout, colori e spaziature definite in `style.css`.

Possibile soluzione:

Correggere il link CSS da `/style.css` a `style.css`.

## F. JSON/API opzionale

### Esercizio 31 - Progettare data.json

Difficoltà: Intermedio | Tempo stimato: 25 min

**Obiettivo.** Separare dati e HTML per un sito statico più ordinato.

#### Consegna.

- 1 Scegli una lista di dati coerente con il tema.
- 2 Crea data.json con almeno 5 elementi.
- 3 Ogni elemento deve avere almeno 3 campi.

**File coinvolti.** data.json

#### Vincoli.

- Il JSON deve essere valido.
- Usa virgolette doppie.
- Non mettere commenti dentro JSON.

#### Soluzione / esempio di soluzione

```
[
 {
 "titolo": "Inception",
 "categoria": "Fantascienza",
 "descrizione": "Un film sui sogni e sui livelli della percezione."
 },
 {
 "titolo": "Interstellar",
 "categoria": "Fantascienza",
 "descrizione": "Un viaggio spaziale alla ricerca di un nuovo futuro."
 },
 {
 "titolo": "The Social Network",
 "categoria": "Biografico",
 "descrizione": "La nascita di Facebook raccontata come dramma tecnologico."
 },
 {
 "titolo": "Whiplash",
 "categoria": "Drammatico",
 "descrizione": "La tensione tra talento, disciplina e ossessione."
 },
 {
 "titolo": "Arrival",
 "categoria": "Fantascienza",
 "descrizione": "Un racconto su linguaggio, tempo e comunicazione."
 }
]
```

### Esercizio 32 - Renderizzare card da data.json

Difficoltà: Avanzato | Tempo stimato: 40 min

**Obiettivo.** Usare fetch per caricare dati locali e mostrarli nella pagina.

#### Consegna.

- 1 Crea un contenitore nel file HTML.

- 2 Collega script.js.
- 3 Carica data.json con fetch.
- 4 Crea una card per ogni elemento.

**File coinvolti.** index.html, script.js, data.json

### Vincoli.

- Questo e extra: chi non ha finito CSS/Pages deve prima finire quelli.
- Il sito deve essere testato su GitHub Pages o con server locale.

### Soluzione / esempio di soluzione

```
<!-- index.html -->
<section class="container my-5">
 <h2 class="mb-4">Catalogo</h2>
 <div id="catalogo" class="row g-4"></div>
</section>

<script src="script.js"></script>

// script.js
fetch("data.json")
 .then(response => response.json())
 .then(data => {
 const catalogo = document.querySelector("#catalogo");

 data.forEach(elemento => {
 catalogo.innerHTML += `
 <div class="col-md-4">
 <article class="card h-100">
 <div class="card-body">
 <h3 class="card-title">${elemento.titolo}</h3>
 <p class="text-muted">${elemento.categoria}</p>
 <p class="card-text">${elemento.descrizione}</p>
 </div>
 </article>
 </div>
 `;
 });
 })
 .catch(error => {
 console.error("Errore nel caricamento dei dati:", error);
 });
```

## Esercizio 33 - Documentare dati/API

*Difficoltà: Intermedio | Tempo stimato: 25 min*

**Obiettivo.** Spiegare struttura e uso dei dati nel progetto.

### Consegna.

- 1 Crea docs/api.md.
- 2 Documenta data.json o API usata.
- 3 Inserisci tabella dei campi ed esempio JSON.

**File coinvolti.** docs/api.md

### Vincoli.

- Se usi una API esterna, indica endpoint e fonte.
- Se usi data.json, documenta comunque il formato.

### Soluzione / esempio di soluzione

```
Documentazione dati
```



## File dati

Il progetto usa il file `data.json` per mostrare il catalogo nella homepage.

## Struttura

```
| Campo | Tipo | Descrizione |
|---|---|---|
| `titolo` | string | Titolo dell'elemento |
| `categoria` | string | Categoria o genere |
| `descrizione` | string | Descrizione breve mostrata nella card |
```

## Esempio

```
```json  
{  
  "titolo": "Inception",  
  "categoria": "Fantascienza",  
  "descrizione": "Un film sui sogni e sui livelli della percezione."  
}  
```
```

## Uso nel sito

Il file viene caricato da `script.js` tramite `fetch("data.json")`.  
Ogni elemento viene trasformato in una card Bootstrap nella sezione Catalogo.

## Esercizio 34 - Fallback se il JSON non si carica

Difficoltà: Avanzato | Tempo stimato: 30 min

**Obiettivo.** Gestire l'errore di caricamento dati in modo visibile all'utente.

### Consegna.

- 1 Modifica script.js.
- 2 Se fetch fallisce, mostra un messaggio nella pagina.
- 3 Non limitarti a console.error.

**File coinvolti.** script.js

### Vincoli.

- Il messaggio deve apparire nel DOM.
- Usa un alert Bootstrap o una classe CSS simile.

### Soluzione / esempio di soluzione

```
fetch("data.json")
 .then(response => {
 if (!response.ok) {
 throw new Error("Risposta non valida");
 }
 return response.json();
 })
 .then(data => {
 const catalogo = document.querySelector("#catalogo");

 data.forEach(elemento => {
 catalogo.innerHTML += `
 <div class="col-md-4">
 <article class="card h-100">
 <div class="card-body">
 <h3 class="card-title">${elemento.titolo}</h3>
 <p class="card-text">${elemento.descrizione}</p>
 </div>
 </article>
 </div>
 `;
 });
 })
 .catch(error => {
```

```
const catalogo = document.querySelector("#catalogo");
catalogo.innerHTML = `
 <div class="col-12">
 <div class="alert alert-danger" role="alert">
 Impossibile caricare i dati. Controllare il file data.json.
 </div>
 </div>
`;
console.error(error);
});
```

## G. Sprint finale

### Esercizio 35 - Sprint finale di consegna

*Difficoltà: Finale | Tempo stimato: 45-60 min*

**Obiettivo.** Portare il progetto a uno stato consegnabile.

#### Consegna.

- 1 Scegli 5 correzioni prioritarie.
- 2 Lavora solo su quelle.
- 3 Aggiorna README e CHANGELOG.
- 4 Pubblica su GitHub Pages.
- 5 Fai commit finale.

**File coinvolti.** tutta la repository

#### Vincoli.

- Non iniziare funzionalità nuove se il sito non è pubblicato.
- Non lasciare la repo sporca.
- Il link Pages deve funzionare.

### Soluzione / esempio di soluzione

Checklist finale:

- [x] `index.html` nella root
- [x] `style.css` collegato
- [x] layout leggibile
- [x] responsive base
- [x] immagini visibili
- [x] README aggiornato
- [x] CHANGELOG aggiornato
- [x] docs minime presenti
- [x] GitHub Pages attivo
- [x] link Pages nel README

Commit finale:

```
```bash
git status
git add .
git commit -m "prepara consegna progetto web"
git push
git status
```
```

Output desiderato finale:

```
```text
nothing to commit, working tree clean
```
```